

2-3-4 TREE CONSTRUCTION

Step-by-Step Insertion and 4-Node Splitting

Dataset: {50, 40, 60, 30, 70, 20, 80}

Presented by: P. Dinesh Karthik

Register No: 192525268

Course Code: CSA0302

Course: Data Structures

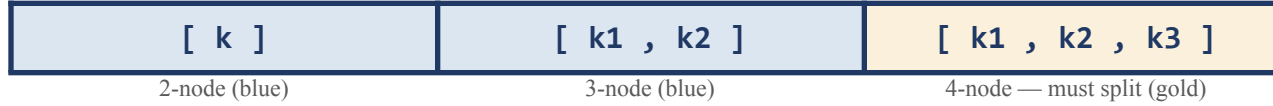
What is a 2-3-4 Tree?

- A self-balancing search tree in which every node can hold 1, 2, or 3 keys and have 2, 3, or 4 children respectively.
- All leaf nodes appear at the same depth — the tree is always perfectly height-balanced.
- It is the conceptual ancestor of the Red-Black Tree (each 2-3-4 tree maps to an equivalent red-black tree).
- Keys inside a node are kept in sorted order; a node with k keys has exactly $k+1$ children.

Why it matters

- Guarantees $O(\log n)$ search, insertion, and deletion because the tree never becomes skewed.

Node Types in a 2-3-4 Tree



- 2-node: 1 key, 2 children — behaves like a normal BST node.
- 3-node: 2 keys, 3 children — keys split the range into 3 parts.
- 4-node: 3 keys, 4 children — this node is "full" and must be split before it can accept another key.

Top-Down Insertion Algorithm

- Start at the root and search downward for the correct leaf position for the new key, exactly like a BST search.
- **Proactive splitting rule: while descending, if the current node is already a 4-node, split it immediately — before moving further down.**
- Splitting a 4-node [a, b, c]: the middle key b moves up into the parent; a and c become two separate 2-nodes (the new children of b).
- If the 4-node being split is the root, the middle key becomes a brand-new root, and the tree grows one level taller.
- Once splitting is done (if needed), the new key is inserted into the appropriate 2-node or 3-node leaf reached during the descent.

This proactive splitting guarantees we never insert into a full 4-node — a single downward pass suffices.

Dataset & Insertion Order

Insert sequence: 50 -> 40 -> 60 -> 30 -> 70 -> 20 -> 80

- Each key is inserted one at a time, applying the top-down splitting rule described on the previous slide.
- **Watch step 4 (inserting 30) closely — this is where the only 4-node split occurs in this sequence.**

Step 1: Insert 50

- Tree is empty, so 50 becomes the root as a single-key 2-node.



Root: [50]

Step 2: Insert 40

- Root [50] is a 2-node (not full) — no split needed.
- 40 is simply added into the root, which becomes a 3-node.

[40 , 50]

Root: [40, 50]

Step 3: Insert 60

- Root [40, 50] is a 3-node (not full) — no split needed yet.
- 60 is added into the root, which now becomes a 4-node (3 keys) — it is FULL.

[40 , 50 , 60]

Root: [40, 50, 60] -> this is now a 4-node and must split on the NEXT insertion that passes through it.

Step 4: Insert 30 — 4-Node Split Occurs

- **Before descending, the algorithm checks the root: [40, 50, 60] is a 4-node, so it MUST be split first.**
- Middle key 50 moves up to become the new root. 40 and 60 separate into two 2-node children.
- Now the (unchanged) key 30 continues its descent: $30 < 50$, so we go to the left child [40]; it is a 2-node, so 30 is inserted directly.

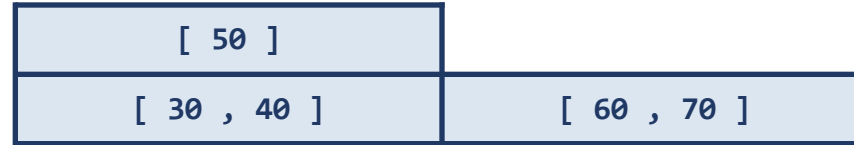
BEFORE SPLIT: [40, 50, 60] (root, 4-node - FULL)

AFTER SPLIT: 50 <- promoted to new root
 / \
 [40] [60]

AFTER INSERTING 30: 50
 / \
 [30, 40] [60] <- 30 inserted into left child

Step 5: Insert 70

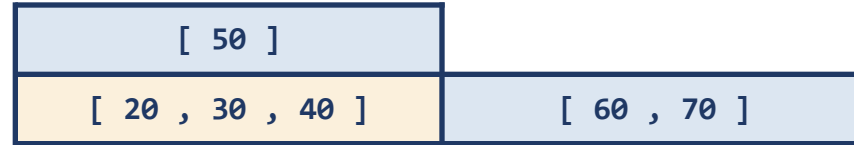
- Root [50] is a 2-node — fine, continue descending.
- 70 > 50, go right to leaf [60]; it is a 2-node (not full) — 70 is inserted directly, forming [60, 70].



Left child: [30, 40] Right child: [60, 70]

Step 6: Insert 20

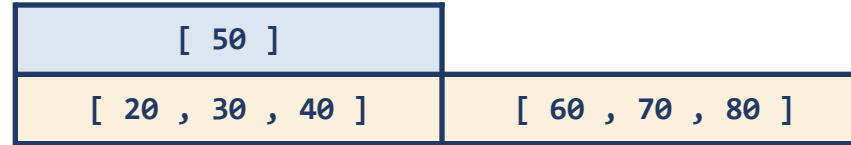
- Root [50] is a 2-node — continue descending.
- $20 < 50$, go left to [30, 40]; it is a 3-node (not full) — 20 is inserted directly, forming [20, 30, 40].



Left child becomes a 4-node: [20, 30, 40] (full, but no further insertion goes through it)

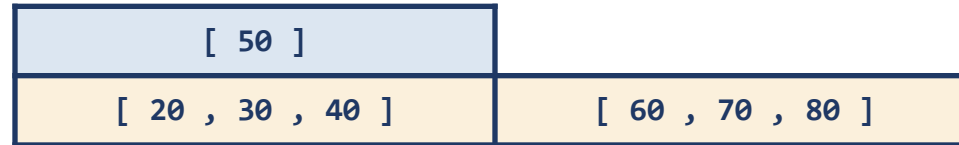
Step 7: Insert 80

- Root [50] is a 2-node — continue descending.
- 80 > 50, go right to [60, 70]; it is a 3-node (not full) — 80 is inserted directly, forming [60, 70, 80].



Both leaves are now 4-nodes, but the insertion sequence ends here — no further splits are triggered.

Final 2-3-4 Tree



- Root: [50] (2-node, 2 children)
- Left child: [20, 30, 40] (4-node — full)
- Right child: [60, 70, 80] (4-node — full)
- **Only ONE split occurred during the entire sequence — while inserting 30, when the root [40, 50, 60] was found to be a 4-node.**

Key Takeaways

- A 2-3-4 tree stays perfectly balanced because splits always happen bottom-up in effect, but are performed top-down in a single pass.
- Splitting a 4-node always promotes its middle key to the parent, keeping every leaf at the same depth.
- Proactive ("on the way down") splitting means insertion never needs a second pass or backtracking.
- If the root itself is a 4-node, splitting it is the only way the tree height increases.
- In this dataset, only the root split once (at key 30) — all other insertions landed in non-full nodes.
- 2-3-4 trees underlie Red-Black Trees, used in many real-world library implementations (e.g., C++ `std::map`, Java `TreeMap`).

Thank You

P. Dinesh Karthik | Reg No: 192525268
CSA0302 - Data Structures